

RAPPORT DE CONDUITE DE PROJET

AI ENGINEERING

Développement du POC d'un agent de triage médical

Centre Hospitalier Saint-Aurélien (CHSA)

Mission réalisée par : [Raymond Francius]

Rôle : AI Engineer (junior)

Durée de la mission : 4 semaines

Date : 16 août 2026

 Apprenant - Promotion 09-2025 : Engineering Intelligence Artificielle (AI) — OpenClassrooms
 Portfolio GitHub : https://github.com/RemDev-AI/OpenClassRooms_Projets_AI_Engineer
 Projet personnel - Agent de triage médical CHSA : <https://github.com/RemDev-AI/medical-triage-agent-ai-poc>

Sommaire

1. Contexte et analyse des besoins
2. Audit de la solution data existante
3. Identification d'une solution technique cible
4. Stratégie de mise en œuvre et d'industrialisation
5. Contrôle et suivi du projet
6. Conclusion & recommandations
7. Annexes

1. Contexte et analyse des besoins

1.1 Présentation de l'organisation et du contexte

Le Centre Hospitalier Saint-Aurélien (CHSA) est un grand hôpital public français dont le service des urgences fait face à une surcharge structurelle, particulièrement marquée aux heures de pointe. Cette tension se traduit par des temps d'attente prolongés et un risque réel de retard dans l'identification des cas critiques, dans un contexte où les effectifs infirmiers dédiés au triage sont insuffisants.

- **Secteur d'activité** : santé publique, établissement hospitalier de grande taille, service d'urgences à fort flux patient.
- **Niveau de maturité IA / MLOps** : faible à ce jour - aucun système d'aide au triage automatisé n'est en place ; le CHSA aborde ce POC comme sa première expérimentation IA en contexte clinique.
- **Enjeux business et humains** : réduction du temps d'attente, sécurisation des cas graves non détectés rapidement, soulagement de la charge du personnel soignant, amélioration de la traçabilité des décisions de triage pour les audits médicaux.
- **Contraintes fortes** : criticité vitale des décisions (l'erreur a un coût humain), cadre réglementaire strict (RGPD, données de santé, potentiel encadrement type dispositif médical), exigence d'explicabilité, contraintes d'intégration au système d'information hospitalier (SIH) existant, ressources GPU limitées imposant un modèle compact dans un premier temps.

1.2 Collecte et analyse du besoin métier

Le besoin exprimé par le CHSA a été formalisé sous la forme d'une mission cadrée en 4 semaines, avec un objectif de démonstration de faisabilité (POC) plutôt que de mise en production complète.

Parties prenantes impliquées

Partie prenante	Rôle dans le projet
Direction du CHSA / service des urgences	Sponsor métier, validation clinique et acceptabilité du système
Personnel infirmier / IOA (Infirmier d'Orientation et d'Accueil)	Utilisateur final, référentiel des protocoles de triage, validation des décisions
DPO / RSSI de l'établissement	Conformité RGPD, sécurité des données de santé
Équipe IT / DSI du CHSA	Intégration au système d'information hospitalier (SIH), infrastructure
AI Engineer (moi-même)	Conception, développement et évaluation du POC technique

Note : dans le cadre d'un POC, la validation clinique reste un jalon simulé/théorique - un déploiement réel nécessiterait un comité médical et une procédure d'homologation formelle.

Méthode de recueil du besoin

- Analyse du brief de mission transmis par le CHSA (cahier des charges fonctionnel et feuille de route sur 4 semaines).
- Revue de la littérature scientifique sur les performances des LLM en contexte médical (référence à l'étude JAMA Network Open sur les capacités diagnostiques des modèles de langage).
- Identification des corpus de référence disponibles pour structurer un dataset médical bilingue (MediQA, FrenchMedMCQA, MedQuAD, UltraMedical-Preference).
- Formalisation d'un backlog produit découpé en 4 sprints hebdomadaires (préparation des données, SFT, DPO, déploiement/validation).

Objectifs techniques et business visés

- Automatiser une première évaluation du niveau de priorité (urgence maximale / modérée / différée) via un questionnaire adaptatif.
- Fournir des recommandations explicables et traçables, exploitables comme aide à la décision - jamais comme substitut au jugement clinique.
- Démontrer la faisabilité technique d'un pipeline complet : collecte de données → fine-tuning (SFT + LoRA) → alignement par préférences (DPO) → déploiement via vLLM → CI/CD.
- Poser les bases d'une architecture évolutive vers des modèles plus grands (32B+) en cas de succès du POC.

Contraintes réglementaires, éthiques et de sécurité

- RGPD : les données de santé constituent une catégorie particulière (article 9) nécessitant anonymisation stricte et base légale claire.
- Principe de non-substitution : l'agent oriente, il ne diagnostique pas ; une escalade humaine doit rester systématique en cas d'incertitude ou de cas grave.
- Explicabilité : chaque recommandation doit être justifiée pour être auditable par les équipes médicales.
- Sécurité : hébergement et flux de données conformes aux exigences d'un établissement de santé (traçabilité des accès, chiffrement).

Hiérarchisation des besoins (matrice impact / effort)

Besoin	Impact clinique/business	Effort technique	Priorité
Évaluation fiable du niveau de priorité	Très élevé	Élevé	P0
Questionnaire adaptatif de collecte des symptômes	Élevé	Moyen	P0
Explicabilité des recommandations	Élevé	Moyen	P0
Traçabilité des interactions (audit)	Élevé	Faible	P1
Intégration au SIH existant	Moyen (POC) / Élevé (prod)	Élevé	P2
Passage à un modèle 32B+	Moyen (à ce stade)	Élevé	P3 (post-POC)

2. Audit de la solution data existante

Le CHSA ne disposait d'aucune solution IA de triage préalable. Cette section décrit donc la solution proposée dans le cadre du POC, en l'auditant comme si elle constituait la brique existante à évaluer avant projection à l'échelle.

2.1 Solution proposée - architecture réellement implémentée

Contrairement à un simple cadrage théorique, le POC a été intégralement implémenté et versionné dans le dépôt GitHub [RemDev-AI/medical-triage-agent-ai-poc](#), avec une architecture applicative complète (backend, frontend, pipeline d'entraînement, CI/CD) et des artefacts publiés sur Hugging Face (modèles et datasets).

Architecture applicative

- **Backend (FastAPI)** : application Python structurée en modules - api (routes triage, inference, auth, audit, health, monitoring, middlewares de sécurité et de logging), llm (chargement du modèle, quantization, moteur d'inférence vLLM, construction de prompts, moteur de triage), anonymization (module dédié basé sur Presidio + spaCy), monitoring (latence, GPU, alerting, audit_store, request_tracker), datasets et training.
- **Frontend (Streamlit)** : application streamlit_app avec pages dédiées (Accueil, Triage, Historique, Métriques, Admin), composants réutilisables (formulaire patient, résultat de triage, cartes de métriques, historique) et services d'appel à l'API (triage_api, auth_client, metrics_api, history_api).
- **Conteneurisation** : un Dockerfile dédié pour le backend et deux Dockerfile pour le frontend (dont un spécifique Hugging Face Spaces), orchestrés en local via [docker-compose.yml](#).
- **CI/CD** : pipeline GitHub Actions effectivement mis en place avec des workflows séparés - [backend-ci.yml](#), [frontend-ci.yml](#), [docker-build-push.yml](#), [deploy-huggingface.yml](#), [security-audit.yml](#) - et un [dependabot.yml](#) pour la veille des dépendances.

Flux de données réellement mis en œuvre

- Téléchargement et standardisation des corpus bruts ([dataset_registry.py](#), [download_datasets.py](#), [standardize_datasets.py](#)) : MediQA (ainsi que ses variantes MCQ mono/multi-réponses), MedQuAD, UltraMedical-Preference - chacun converti en un schéma standardisé (raw/standardized).
- Construction des jeux de données d'entraînement finaux (processed/sft et processed/dpo), chacun découpé en 4 splits : train, validation, test et clinical_eval - ce dernier split dédié spécifiquement à l'évaluation clinique du modèle.
- Publication du dataset final sur Hugging Face Hub ([RemDev-AI/medical-triage-agent-ai-poc-datasets](#)), avec configuration explicite des splits sft et dpo dans le README (configs YAML), permettant un chargement direct via la librairie datasets.
- Chaque interaction de triage passe par le module d'anonymisation (détection PII via des patterns dédiés + Presidio Analyzer/Anonymizer) avant tout traitement, avec un audit_logger dédié pour la traçabilité.

Outils et technologies effectivement retenus

Brique	Outil / technologie	Élément constaté dans le dépôt
Modèle de base	Qwen3-1.7B	Fine-tuné en SFT puis DPO, checkpoints publiés sur Hugging Face
Fine-tuning	SFT + LoRA (PEFT)	training/sft/train_sft.py , training/lora/peft_setup.py , checkpoint-sft-43
Alignement	DPO (TRL)	training/dpo/train_dpo.py , dpo_config.yaml , checkpoint-dpo-32 avec modèle de référence (ref/)
Entraînement accéléré	Unsloth	Patch appliqué en environnement Colab pour un fine-tuning ~2x plus rapide sur GPU contraint
Environnement d'entraînement	Google Colab (GPU T4, 15,6 Go VRAM) + Kaggle	Modules colab/ et kaggle/ dédiés (détection GPU, synchronisation de checkpoints, login HF)
Anonymisation RGPD	Microsoft Presidio + spaCy	presidio_analyzer.py , presidio_anonymizer.py , spacy_setup.py , pii_patterns.py
Serving / inférence	vLLM	llm/inference/vllm_engine.py , endpoint exposé via Hugging Face Space
Hébergement final	Hugging Face Spaces	API Swagger exposée publiquement (voir section 4)
CI/CD	GitHub Actions	5 workflows dédiés (build/test backend & frontend, build Docker, déploiement HF, audit sécurité)

2.2 Évaluation de l'adéquation aux besoins

Critères d'analyse

Critère	Évaluation	Commentaire
Performance clinique	Mesurée	Un pipeline d'évaluation clinique dédié existe (clinical_eval_runner.py , clinical_metrics.py , clinical_thresholds.py , safety_evaluator.py , hallucination_detector.py , dangerous_recommendation_detector.py) et produit un rapport structuré (evaluation_report.json/.md) publié sur Hugging Face pour le modèle qwen3-1.7b-dpo
Robustesse	Testée	Suite de tests dédiée : tests/performance/test_robustness.py , test_latency.py , ainsi que tests/security (contrôle d'accès, endpoints, injections)
Sécurité des données	Bonne	Anonymisation Presidio/spaCy en amont de tout traitement, audit_logger dédié, middleware d'authentification et de sécurité au niveau API
Coût	Faible	Entraînement réalisé sur GPU gratuit (Colab T4) grâce à LoRA et à l'accélération Unsloth
Maintenance / évolutivité	Bonne	5 pipelines CI/CD actifs, synchronisation automatique des checkpoints vers Hugging Face Hub (colab_checkpoint_sync.py)
Monitoring en production	Partiellement construit	Modules monitoring/ présents (gpu_monitor , latency_monitor , inference_monitor , alerting , request_tracker) mais non encore branchés à un tableau de bord externe (Prometheus/Grafana)

Écarts et limites identifiés

- Absence, à ce stade, d'un déclenchement automatique du réentraînement : les checkpoints sont synchronisés vers le Hub, mais le lancement d'un nouveau cycle SFT/DPO reste manuel (exécution du notebook Colab).
- L'entraînement dépend d'un GPU T4 gratuit à VRAM limitée (15,6 Go), avec des contraintes explicites (BF16 natif désactivé et forcé via patch Unsloth), ce qui borne la taille du modèle entraînable en l'état.
- Le monitoring existant (gpu_monitor, latency_monitor, inference_monitor) n'est pas encore relié à un outil de visualisation externe type Grafana : les métriques sont collectées mais pas encore exposées sous forme de tableau de bord temps réel.
- L'intégration au système d'information hospitalier (SIH) du CHSA reste hors périmètre du POC : l'API est exposée publiquement via Hugging Face Space, sans connecteur SIH dédié à ce stade.
- Le split clinical_eval, bien que présent dans les datasets SFT et DPO, nécessite une validation par des professionnels de santé pour garantir la pertinence du jeu de référence utilisé par le safety_evaluator.

3. Identification d'une solution technique cible

3.1 Comparatif d'approches techniques

Approche	Avantages	Inconvénients
LLM généraliste + RAG (base de connaissances / protocoles)	Rapide à mettre en œuvre, traçable (sources citées), pas de réentraînement lourd	Dépend de la qualité de la base de connaissances, moins spécialisé sur le vocabulaire clinique
Fine-tuning SFT + LoRA (retenu pour le POC)	Spécialisation efficace, coût GPU maîtrisé, bon compromis performance/ressources	Nécessite un dataset labellisé de qualité, risque d'oubli catastrophique si mal calibré
Alignement par préférences (DPO)	Aligne le comportement sur des préférences cliniques validées, réduit les réponses inadéquates	Nécessite des paires de préférences fiables et représentatives, sensible au biais des annotateurs
Modèle spécialisé de grande taille (32B+)	Meilleure capacité de raisonnement médical	Coût d'inférence et d'entraînement élevé, non justifié tant que le POC n'est pas validé

Le choix retenu pour le POC - modèle compact Qwen3-1.7B + SFT/LoRA + DPO - constitue un compromis pragmatique entre rapidité d'itération, maîtrise des coûts et capacité à démontrer la valeur clinique du concept, avant d'engager les ressources plus importantes qu'exigerait un modèle de 32B+ paramètres.

3.2 Schéma d'architecture cible

L'architecture cible correspond à celle effectivement implémentée dans le dépôt (cf.

[images/Architecture_POC_Agent_Triage_Médical.png](#)), organisée en couches :

- **Couche d'interaction patient** : application Streamlit (streamlit_app) avec formulaire patient adaptatif (patient_form.py), affichage du résultat de triage (triage_result.py) et historique des interactions (history_table.py).
- **Couche API / orchestration** : backend FastAPI exposant les routes triage, inference, auth, audit, health et monitoring, avec middlewares dédiés (authentification, sécurité, logging) en amont de chaque requête.
- **Couche modèle** : moteur de triage (triage_engine.py) s'appuyant sur le prompt_builder.py et le moteur d'inférence vLLM (vllm_engine.py), qui charge le modèle Qwen3-1.7B fine-tuné (adaptateurs LoRA SFT puis DPO fusionnés via merge_lora_adapter.py).
- **Couche anonymisation** : toute donnée patient transite par le module anonymization (détection de PII, analyse et anonymisation Presidio, journalisation dédiée via audit_logger) avant tout traitement ou stockage.
- **Couche données** : datasets versionnés (raw → standardized → processed) et publiés sur Hugging Face Hub ; journal d'audit (audit_store.py) pour la traçabilité de chaque décision de triage.
- **Couche monitoring** : modules dédiés au suivi de la latence, de la charge GPU et des inférences (latency_monitor, gpu_monitor, inference_monitor), avec système d'alerting intégré.
- **Couche déploiement** : conteneurisation Docker (backend + frontend), publication automatisée sur Hugging Face Spaces via le workflow deploy-huggingface.yml, API documentée et accessible via Swagger.

- **Couche future - intégration SIH (hors périmètre du POC actuel)** : connecteur à ajouter vers le système d'information hospitalier du CHSA pour la transmission des cas au personnel infirmier.

3.3 Justification des choix technologiques

- **Qwen3-1.7B-Base** : modèle open-weight compact, permettant un cycle d'itération rapide et une empreinte GPU compatible avec un POC en 4 semaines.
- **LoRA** : adaptation à faible rang qui limite le nombre de paramètres entraînés, réduisant le coût de calcul et le risque de sur-ajustement sur un dataset de taille modeste.
- **DPO** : méthode d'alignement par préférences directe, plus stable et moins coûteuse à mettre en œuvre qu'un RLHF classique, adaptée à un contexte de ressources limitées.
- **vLLM** : moteur d'inférence optimisé pour la latence et le débit, pertinent dans un contexte d'urgences où le temps de réponse est critique.
- **GitHub Actions** : solution de CI/CD légère, largement adoptée, suffisante pour automatiser les tests et déploiements dans le cadre d'un POC.

3.4 Méthodologie d'identification et de priorisation des cas d'usage

Les cas d'usage ont été évalués selon une matrice valeur clinique / effort de mise en œuvre, en croisant l'impact potentiel sur la sécurité des patients et la complexité technique de réalisation.

Cas d'usage	Valeur clinique	Effort	Priorité
Triage initial par questionnaire adaptatif	Très élevée	Moyen	1
Explication de la recommandation au patient	Élevée	Faible	1
Escalade automatique en cas d'incertitude	Très élevée	Faible	1
Intégration temps réel au SIH	Élevée	Élevé	2
Réentraînement automatique périodique du modèle	Moyenne à ce stade / Élevée à terme	Élevé	2
Extension multilingue au-delà du français/anglais	Faible à ce stade	Élevé	3

4. Stratégie de mise en œuvre et d'industrialisation

4.1 Proposition de démarche projet

Roadmap de mise en œuvre (4 semaines)

Semaine	Jalon principal	Livrable réellement produit
S1	Préparation et structuration des données	Datasets standardisés (MediQA, MedQuAD, UltraMedical-Preference) + jeux SFT/DPO à 4 splits, publiés sur Hugging Face (medical-triage-agent-ai-poc-datasets)
S2	Fine-tuning supervisé (SFT + LoRA)	checkpoint-sft-43 (LoRA/PEFT, entraîné via Unsloth sur Colab GPU T4), synchronisé sur Hugging Face
S3	Alignement par préférences (DPO)	checkpoint-dpo-32 avec modèle de référence, rapport d'évaluation clinique qwen3-1.7b-dpo (evaluation_report.json/.md)
S4	Déploiement pilote et validation finale	Backend FastAPI + frontend Streamlit conteneurisés, déployés sur Hugging Face Space avec API Swagger publique, 5 pipelines CI/CD actifs

Découpage en étapes techniques

- Développement : préparation des données, scripts d'entraînement SFT/LoRA et DPO.
- Tests : évaluation offline sur jeu de test de référence, revue humaine (human-in-the-loop) sur échantillon de cas.
- Conteneurisation : packaging du modèle et de son environnement d'inférence pour portabilité.
- Intégration : exposition d'un endpoint API compatible avec les futurs connecteurs du SIH.
- Déploiement : mise en ligne de l'endpoint de démonstration via vLLM sur le cloud.
- Monitoring : mise en place de journaux structurés pour la traçabilité et base pour un futur monitoring de production.

Outils envisagés par phase

Phase	Outils réellement utilisés
Préparation des données	dataset_registry.py , download_datasets.py , standardize_datasets.py ; anonymisation via Presidio + spaCy ; publication sur Hugging Face Hub
Entraînement (SFT/DPO)	Transformers, PEFT/LoRA, TRL (DPO), Unsloth (accélération), exécuté via runpy sur Notebook Colab GPU T4 (et environnement Kaggle en alternative)
Suivi d'expérimentation / checkpoints	colab_checkpoint_sync.py - sauvegarde et restauration automatique des checkpoints SFT/DPO depuis/vers Hugging Face Hub
Évaluation clinique	clinical_eval_runner.py , clinical_metrics.py , safety_evaluator.py , hallucination_detector.py , dangerous_recommendation_detector.py
Déploiement	vLLM (inférence), Docker (backend + frontend), Hugging Face Spaces (hébergement, API Swagger publique)
CI/CD	GitHub Actions - backend-ci.yml , frontend-ci.yml , docker-build-push.yml , deploy-huggingface.yml , security-audit.yml , dependabot.yml
Démonstration	Application Streamlit multi-pages (Accueil, Triage, Historique, Métriques, Admin)

4.2 Aide à la prise de décision

Mise en place de mécanismes de réentraînement automatique - thématique clé approfondie

Le mécanisme de réentraînement automatique, envisagé au démarrage du POC comme piste d'industrialisation, a depuis été entièrement conçu et implémenté en 4 étapes, sans aucune régression sur les scripts d'entraînement et d'évaluation existants :

- **Détection automatique** : nouveau workflow GitHub Actions `"retraining-detection.yml"` (déclenchement manuel via `workflow_dispatch`) interroge les endpoints authentifiés `"GET /audit/clinical"` et `"GET /monitoring/requests"` exposés par le Space Hugging Face, pour calculer les indicateurs de dérive : désaccords entre le triage_engine et la validation humaine, taux d'erreur et volume de nouveaux cas depuis la dernière demande.
- **Préparation automatique** : lorsque les seuils sont dépassés, le workflow publie un fichier `"retraining_request.json"` sur le même dépôt Hugging Face que les checkpoints (`RemDev-AI/medical-triage-agent-ai-poc-models`), en réutilisant le jeton déjà utilisé par `"deploy-huggingface.yml"` - aucun nouveau secret Hugging Face à créer.
- **Exécution manuelle sur Colab** : la cellule 19 du notebook `"training_pipeline_colab.ipynb"` a été complétée pour lire et afficher la demande de réentraînement avant l'enchaînement `train_sft` → `train_dpo` → `clinical_eval_runner` (cellules inchangées). Cet ajout est purement informatif : en cas d'erreur réseau ou d'absence de demande, il ne bloque jamais l'exécution des cellules suivantes. La décision de lancer effectivement l'entraînement GPU reste manuelle, contrainte imposée par le quota Colab GPU T4 Free (pas d'API pour piloter Colab à distance).
- **Clôture automatique** : une nouvelle cellule, insérée après `clinical_eval_runner`, relit le rapport d'évaluation clinique publié (`"dpo/evaluation_reports/{model_name}/evaluation_report.json"`) et met à jour `"retraining_request.json"` avec le statut final (`completed/failed`), déterminé sur son champ canonique `overall_status` (complété par `safety.thresholds_passed`) ainsi que la référence du checkpoint promu. Le tout en best-effort : toute erreur réseau ou absence de rapport est journalisée sans jamais interrompre le notebook.
- **Non-régression garantie** : les scripts `"train_sft.py"`, `"train_dpo.py"`, `"clinical_eval_runner.py"`, `"safety_evaluator.py"`, `"hallucination_detector.py"`, `"dangerous_recommendation_detector.py"` et `"peft_setup.py"` restent de simples exécutants, sans logique de décision liée au réentraînement - la séparation des responsabilités validée en amont a été préservée de bout en bout.
- **Versioning et rollback** : la structure de checkpoints déjà en place (checkpoint-sft-43, checkpoint-dpo-32, dossiers final/) continue de permettre un rollback immédiat vers une version antérieure en cas de régression détectée par le `safety_evaluator` après déploiement.
- **Limite structurelle identifiée et à surveiller** : le workflow `"deploy-huggingface.yml"` redémarre le Space à chaque déploiement (`restart_space`), ce qui réinitialise les journaux locaux au conteneur (`api_audit_trail.jsonl` et surtout `clinical_audit_trail.jsonl`). L'ordre d'exécution manuel doit donc être respecté : lancer `"retraining-detection.yml"` avant tout déclenchement de déploiement, sous peine de manquer un signal de réentraînement réel.

- **Prochaines étapes d'industrialisation** : faire évoluer le déclenchement du workflow de détection, aujourd'hui manuel (workflow_dispatch), vers un job planifié (schedule) ; raccorder à terme `"deploy-huggingface.yml"` pour un redéploiement automatique de l'endpoint dès qu'une version validée est disponible ; et sécuriser les routes `"/monitoring/*"`, aujourd'hui accessibles sans authentification contrairement à `"/audit/*"`.

Synthèse des risques et opportunités

Risque / Opportunité	Nature	Niveau	Levier d'atténuation
Faux négatif sur un cas critique	Risque	Critique	Priorisation de la sensibilité, escalade humaine systématique en cas de doute
Biais dans les données d'entraînement	Risque	Élevé	Audit des corpus, équilibrage des sources francophones/anglophones
Non-conformité RGPD	Risque	Élevé	Anonymisation dès la collecte, validation DPO/RSSI du CHSA
Dérive du modèle en production (data/concept drift)	Risque	Moyen	Monitoring continu et mécanisme de réentraînement automatique décrit ci-dessus
Réduction du temps d'attente aux urgences	Opportunité	Élevé	Généralisation progressive après validation clinique du POC
Amélioration de la traçabilité pour les audits	Opportunité	Moyen	Journalisation structurée dès la conception

Scénarios budgétaires (estimation indicative)

Poste de coût	POC (4 semaines)	Projection industrielle (Phase 3)
Infrastructure GPU (entraînement)	Faible - modèle 1.7B, LoRA	Élevé - modèles 32B+, entraînement complet ou LoRA à grande échelle
Coût d'inférence	Faible - endpoint unique de démonstration	Modéré à élevé - selon volumétrie patients et disponibilité 24/7
MLOps / CI/CD	Faible - GitHub Actions, un pipeline simple	Modéré - pipelines multi-environnements, tests de charge
Conformité et sécurité	Modéré - anonymisation, audit RGPD initial	Élevé - homologation éventuelle type dispositif médical, audits récurrents

Pour objectiver les ordres de grandeur évoqués ci-dessus, les deux tableaux suivants proposent une estimation chiffrée indicative, phase par phase. Ces montants sont des ordres de grandeur destinés à alimenter la décision d'investissement ; ils devront être affinés lors du chiffrage détaillé (devis fournisseurs cloud, ressources humaines, prestataires réglementaires).

Détail chiffré indicatif - Phase POC (4 semaines)

Poste de coût	Hypothèse retenue	Coût estimé
Infrastructure GPU (entraînement)	Google Colab Pro (GPU T4/A100), 1 mois	~ 12 €
Stockage & versioning (Hugging Face Hub)	Plan gratuit (datasets et checkpoints publics)	0 €
Coût d'inférence (démonstration)	Hugging Face Space CPU basic, 1 mois	~ 9 €
MLOps / CI/CD	GitHub Actions, quota gratuit (dépôt existant)	0 €
Conformité et sécurité	Anonymisation open source (Presidio/spaCy) + 2 j.homme de revue RGPD	~ 900 €
Ressources humaines (développement)	1 ETP Data Scientist/AI Engineer x 4 semaines (20 j. à 450 €/j.)	~ 9 000 €
Total indicatif POC		~ 9 900 - 10 000 €

Détail chiffré indicatif - Projection industrielle (Phase 3, en régime annuel)

Poste de coût	Hypothèse retenue	Coût estimé (annuel)
Infrastructure GPU (entraînement)	Cycles de réentraînement périodiques sur instance cloud A100 à la demande (~40 h/mois)	~ 6 000 - 10 000 €
Coût d'inférence (production 24/7)	1 à 2 instances GPU dédiées (A10/L4) pour disponibilité continue	~ 18 000 - 30 000 €
MLOps / CI/CD	Pipelines multi-environnements, tests de charge, astreinte partielle	~ 8 000 - 12 000 €
Conformité et sécurité	Audits RGPD récurrents et accompagnement réglementaire (dispositif médical)	~ 20 000 - 40 000 €
Monitoring et supervision	Stack d'observabilité (journalisation, alerting, tableaux de bord)	~ 5 000 - 8 000 €
Ressources humaines	Équipe élargie (AI Engineer, MLOps, référent clinique), ~2 à 3 ETP	~ 180 000 - 270 000 €
Total indicatif Phase 3 (annuel)		~ 240 000 - 370 000 €

Ces estimations supposent un déploiement à l'échelle d'un seul établissement pilote ; une généralisation à plusieurs sites du CHSA ou à d'autres établissements ferait varier sensiblement les postes d'inférence, de MLOps et de conformité. Elles sont non contractuelles et doivent être validées avec les équipes achats/infrastructure et le DPO/RSSI du CHSA.

Indicateurs de succès (KPI)

KPI	Type	Cible indicative
Sensibilité sur cas critiques (rappel)	Technique / clinique	Maximisée en priorité - tolérance faible aux faux négatifs graves
Taux d'escalade humaine appropriée	Clinique	À calibrer avec les équipes IOA
Latence de réponse de l'agent	Technique	Compatible avec le rythme du flux d'urgences
Taux d'acceptabilité par le personnel soignant	Business / organisationnel	Mesuré via retours qualitatifs en phase pilote
Taux de désaccord agent / décision finale humaine	Technique (pilote le réentraînement)	Suivi dans le temps, alimente le pipeline de réentraînement

Impacts potentiels et leviers d'atténuation

- **Impact légal / réglementaire** : un déploiement en production dépasserait le cadre du POC et pourrait relever d'un encadrement de type dispositif médical (MDR) - anticiper une revue juridique avant tout passage à l'échelle.
- **Impact organisationnel** : nécessité d'accompagner le personnel soignant dans l'appropriation de l'outil (formation, communication sur les limites de l'agent).
- **Biais algorithmique** : vigilance sur la représentativité des corpus francophones/anglophones utilisés, afin d'éviter une sous-performance sur certains profils de patients.
- **Faille de sécurité** : les données de santé étant particulièrement sensibles, un chiffrement de bout en bout et un contrôle d'accès strict sont non négociables dès la phase pilote.
- **Latence** : un temps de réponse trop long dégraderait l'expérience aux urgences ; le choix de vLLM et d'un modèle compact vise justement à limiter ce risque.

4.3 Architecture Technique type MLOps

La solution développée dans le cadre du POC repose sur un mécanisme de réentraînement conçu sur mesure (workflows GitHub Actions et notebook Colab, cf. section 4.2), adapté aux contraintes d'un pilote mono-établissement. En vue d'une projection industrielle à plus grande échelle (généralisation à plusieurs services ou établissements, volumétrie accrue, exigences de gouvernance renforcées), une architecture cible s'appuyant sur des briques open source éprouvées permettrait de fiabiliser et d'automatiser davantage ce mécanisme. La **stack open source de référence** envisagée pour cette phase d'industrialisation se compose des éléments suivants :

- **Détection de dérive (Evidently / NannyML)** : surveillance continue du data drift et du concept drift sur les échanges de triage, en remplacement ou en complément du calcul actuel de désaccords triage_engine / validation humaine, avec génération de rapports et d'alertes exploitables comme déclencheur objectif du réentraînement.
- **Orchestration (Airflow / Kubeflow Pipelines)** : remplacement du déclenchement manuel (workflow_dispatch) par une orchestration planifiée et événementielle du pipeline complet - détection de dérive, préparation des données, lancement de l'entraînement SFT/DPO et clôture - sans dépendre d'une exécution manuelle sur notebook Colab.
- **Suivi d'expérimentation et registre de modèles (MLflow)** : traçabilité centralisée des runs SFT/DPO (hyperparamètres, métriques cliniques, rapports de sécurité) et gestion des transitions de statut d'un modèle (staging → production), en complément du rapport evaluation_report.json déjà produit par clinical_eval_runner.
- **Versioning des données et artefacts (DVC)** : traçabilité et reproductibilité renforcées des jeux de données (raw → standardized → processed) et des checkpoints, en complément du versioning déjà assuré par Hugging Face Hub, avec un lien explicite entre une version de dataset et le modèle qui en résulte.
- **Déploiement du modèle réentraîné (Seldon / KServe / BentoML)** : automatisation du redéploiement de l'endpoint dès qu'un modèle validé est disponible, répondant directement à la limite identifiée en 4.2 (raccordement de `deploy-huggingface.yml` à un redéploiement automatique), avec gestion native du rollback en cas de régression détectée par le safety_evaluator.

Cette **stack** constitue une cible d'évolution et non une remise en cause du pipeline déjà implémenté : les briques actuelles (GitHub Actions, notebook Colab, synchronisation Hugging Face) resteraient opérationnelles en phase de transition, chaque composant open source venant remplacer progressivement son équivalent artisanal à mesure que la volumétrie et les exigences de gouvernance justifient l'investissement associé.

4.4 Architecture Financière type FinOps

Le POC ne dispose pas encore d'un suivi financier structuré des coûts d'inférence et d'entraînement : seule une estimation indicative figure dans les scénarios budgétaires (cf. section 4.2). En vue d'une projection industrielle, une démarche **FinOps** outillée permettrait de suivre en continu le coût réel du service - appels API/LLM, GPU d'entraînement et d'inférence, infrastructure Kubernetes - et de le rapprocher de la valeur clinique produite. La **stack open source de référence** envisagée pour cette phase se compose des éléments suivants :

- **Suivi des coûts d'API LLM (LiteLLM ou Helicone)** : proxy unifié devant les appels au modèle (endpoint vLLM déployé sur Hugging Face Space, ou futurs providers externes), avec suivi des tokens et du coût par clé, par utilisateur ou par cas d'usage, budgets et rate-limiting intégrés.
- **Observabilité fine des traces et coûts (Langfuse)** : traçabilité de chaque appel de triage_engine (prompt, tokens, latence, coût), permettant de rapprocher le coût par requête du niveau de priorité déterminé et du taux de désaccord agent / décision humaine déjà suivi comme KPI.
- **Suivi de l'utilisation GPU réelle (Prometheus / DCGM Exporter / Grafana)** : complément du monitoring déjà existant (gpu_monitor, latency_monitor, inference_monitor) - identifié en section 2.2 comme non encore relié à un tableau de bord externe - avec un lien direct entre le coût horaire des instances GPU (entraînement Colab/Kaggle puis inférence en production) et leur taux d'utilisation réel, afin de détecter le sur-provisionnement.
- **Allocation des coûts d'infrastructure Kubernetes (OpenCost / Kubecost)** : pertinent dès lors que le déploiement dépasserait Hugging Face Spaces pour s'orienter vers un cluster Kubernetes dédié (cf. couche SIH à intégrer, section 3.2) - ventilation du coût par namespace, équipe ou établissement en cas de généralisation à plusieurs sites du CHSA.
- **Chiffrage prévisionnel avant déploiement (Infracost)** : estimation du coût cloud en amont, à partir des définitions d'infrastructure as code, pour fiabiliser les scénarios budgétaires de la phase 3 (cf. section 4.2) avant tout provisionnement réel.

Comme pour le mécanisme de réentraînement automatique (section 4.3), cette **stack FinOps** est une cible d'évolution : elle viendrait outiller et fiabiliser le suivi budgétaire actuellement estimatif, sans remettre en cause l'architecture applicative déjà implémentée et versionnée dans le dépôt du POC.

5. Contrôle et suivi du projet

5.1 Tableau de bord de pilotage

Indicateurs de suivi de projet

Indicateur	Suivi
Respect des délais	Suivi hebdomadaire par rapport à la roadmap en 4 sprints
Qualité des données	Contrôle de conformité RGPD et taux d'anonymisation validé à l'issue de S1
Avancement des livrables	Revue de sprint hebdomadaire (dataset, modèle SFT, modèle DPO, endpoint)
Performances du modèle	Suivi des métriques d'évaluation intermédiaires (S2) et finales (S4)
Coûts	Suivi de la consommation GPU réelle vs. budget prévisionnel du POC

Méthodologie de gestion de projet

Le projet a été mené selon une logique agile adaptée à un POC court (4 semaines) : découpage en 4 sprints hebdomadaires alignés sur les phases du pipeline (données, SFT, DPO, déploiement), avec un point d'avancement à l'issue de chaque semaine permettant d'ajuster la trajectoire si nécessaire (approche proche d'un Scrum allégé).

5.2 Outils et process de suivi

Outils de suivi réellement en place

- **Journalisation et audit** : `“audit_logger.py”` (module anonymization) et `“audit_store.py”` (module monitoring), exposés via la route API `“audit.py”`, pour la traçabilité de chaque interaction exigée pour les audits médicaux.
- **Monitoring applicatif** : `“gpu_monitor.py”`, `“latency_monitor.py”`, `“inference_monitor.py”` et `“request_tracker.py”`, avec un système d'alerting.py dédié, exposés via la route `“monitoring.py”` et la route `“health.py”`.
- **Suivi des checkpoints d'entraînement** : synchronisation automatique vers/depuis Hugging Face Hub (`colab_checkpoint_sync.py`, `kaggle_checkpoint_sync.py`), avec restauration du dernier checkpoint disponible par type d'entraînement (SFT/DPO).
- **Rapports d'évaluation clinique** : `“evaluation_report.py”` produit un rapport structuré (JSON + Markdown), publié et consultable directement sur Hugging Face (dossier `evaluation_reports/qwen3-1.7b-dpo`).
- **Piste d'évolution** : Prometheus / Grafana restent à connecter aux modules de monitoring existants pour disposer d'un tableau de bord temps réel, la collecte de métriques brutes étant déjà opérationnelle.

Méthodologie de test et d'évaluation réellement appliquée

- **Tests unitaires** : tests/unit - `"test_anonymization.py"`, `"test_api.py"`, `"test_loaders.py"`.
- **Tests d'intégration** : tests/integration - `"test_pipeline.py"`, `"test_inference_api.py"`, `"test_frontend_backend.py"`.
- **Tests de sécurité** : tests/security - `"test_access_control.py"`, `"test_endpoints.py"`, `"test_injections.py"`, complétés par le workflow CI dédié security-audit.yml.
- **Tests de performance** : tests/performance - `"test_latency.py"`, `"test_robustness.py"`, `"test_audit_trail.py"`.
- **Évaluation clinique offline** : `"clinical_eval_runner.py"` exécuté après SFT et après DPO sur le split clinical_eval dédié, avec détection d'hallucinations (`hallucination_detector.py`) et de recommandations dangereuses (`dangerous_recommendation_detector.py`).
- **Tests API** : documentation et test interactif via l'API Swagger exposée publiquement (remdev-ai-medical-triage-agent-ai-poc-api.hf.space/docs).

6. Conclusion & recommandations

Résumé des choix clés

- Un modèle compact (Qwen3-1.7B) a été retenu pour permettre une validation rapide et peu coûteuse du concept, avant d'envisager un passage à l'échelle.
- La combinaison SFT + LoRA puis DPO permet une spécialisation clinique du modèle tout en maîtrisant les ressources de calcul mobilisées.
- L'architecture proposée intègre, dès le POC, les fondations d'une industrialisation future : CI/CD via GitHub Actions, endpoint optimisé via vLLM, et principes de traçabilité et de réentraînement automatique pensés en amont.
- Le principe de non-substitution au jugement médical et l'escalade humaine systématique en cas d'incertitude ont été posés comme garde-fous structurants de la solution.

Perspectives d'évolution

- Passage à des modèles de plus grande taille (32B+) une fois la valeur clinique du POC démontrée, avec un jeu de données étendu et enrichi.
- Mise en place effective du pipeline de réentraînement automatique décrit en section 4.2, incluant détection de dérive et validation humaine avant chaque mise à jour.
- Intégration progressive et testée avec le système d'information hospitalier du CHSA, au-delà de la simulation d'inférence réalisée en Phase 1.
- Enrichissement de l'architecture par une brique RAG connectée aux protocoles de triage officiels (ex. Manchester Triage Scale) pour renforcer l'explicabilité et la traçabilité clinique.

Prochaines étapes recommandées

- Organiser une revue clinique formelle des résultats du POC avec le comité médical du CHSA.
- Réaliser une étude d'impact réglementaire (RGPD, éventuel encadrement dispositif médical) avant tout passage en environnement pilote.
- Définir précisément les seuils de dérive et les KPI cliniques qui déclencheront les futurs cycles de réentraînement automatique.
- Planifier une phase pilote encadrée, avec un sous-ensemble de cas à faible risque, avant généralisation.

7. Annexes

Dépôt et code

- Dépôt GitHub du projet : “github.com/RemDev-AI/medical-triage-agent-ai-poc” (backend FastAPI, frontend Streamlit, pipelines de training, workflows CI/CD).
- Notebook Colab (GPU T4, gratuit) - Training Pipeline : “[backend/app/training/colab/training_pipeline_colab.ipynb](https://colab.research.google.com/github/RemDev-AI/medical-triage-agent-ai-poc/blob/main/backend/app/training/colab/training_pipeline_colab.ipynb)”.
- **README.md** du projet et documentation technique interne : backend/app/docs (1-architecture.md, 2-rgpd.md, 3-training.md, 4-deployment.md, 5-api.md, 6-modal.md, 7 et 8-final_report.md).

Modèles et datasets (Hugging Face Hub)

- Dataset final : “huggingface.co/datasets/RemDev-AI/medical-triage-agent-ai-poc-datasets” (splits sft et dpo, chacun en train/validation/test/clinical_eval).
- Modèles - checkpoints SFT : “huggingface.co/RemDev-AI/medical-triage-agent-ai-poc-models/tree/main/sft/checkpoints/checkpoint-sft-43”.
- Modèles - checkpoints DPO : “huggingface.co/RemDev-AI/medical-triage-agent-ai-poc-models/tree/main/dpo/checkpoints/checkpoint-dpo-32” (avec modèle de référence associé).
- Modèles finaux (SFT et DPO) : dossiers sft/final et dpo/final du même dépôt Hugging Face.
- Rapport d'évaluation clinique du modèle qwen3-1.7b-dpo : dossier dpo/evaluation_reports/qwen3-1.7b-dpo (**evaluation_report.json** et **.md**).

Déploiement

- API de démonstration déployée sur Hugging Face Space, documentation interactive Swagger : “remdev-ai-medical-triage-agent-ai-poc-api.hf.space/docs”.
- Script de déploiement vers le Space : “**scripts/deploy_to_hf_space.sh**”, orchestré par le workflow “**deploy-huggingface.yml**”.

Autres ressources

- Schéma d'architecture du POC : “images/Architecture_POC_Agent_Triage_Médical.png”.
- **Référence bibliographique** : étude JAMA Network Open sur les performances diagnostiques des modèles de langage en contexte médical (consultée en phase de cadrage).
- Datasets sources mobilisés : MediQA (et variantes MCQ), MedQuAD, UltraMedical-Preference.